

Comparative Analysis of a Custom k-Nearest Neighbors and Weighted k-Nearest Neighbors Classifier on a Model Simulation Dataset

Roshna George, Vakalapudi Sanjeev, Nithilan Rameshkumar

Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, Bengaluru, India

1 Methodology

All experiments were implemented in Python 3.11 using NumPy and pandas for data handling, scikit-learn 1.5.2 for the reference classifier, and matplotlib for visualization.

1.1 Dataset and Preprocessing

The project dataset `simulation_500.csv` stores results of a simulation in which four model-building methods are evaluated over several prediction problems. It contains 2,000 samples described by seven attributes: `method`, `bias`, `mae`, `rmse`, `spearman`, `accuracy`, and `rules`. The `method` attribute is categorical and was converted into numeric values by label encoding (each unique value mapped to 0, 1, 2, ...); a one-hot encoding function was also implemented for comparison. Because the data is machine-generated, no missing values were observed, yet an imputation module was developed that fills incomplete columns with the chosen central tendency: the mean, the median, or the mode.

A binary classification problem was derived by thresholding the continuous `accuracy` attribute: samples with $\text{accuracy} \geq 0.5$ are assigned to class 1 and the remaining samples to class 0. The `accuracy` column itself is excluded from the feature set to prevent target leakage, leaving six features (`method`, `bias`, `mae`, `rmse`, `spearman`, `rules`). As shown in Table 1, the target is imbalanced (1,543 samples of class 1 versus 457 of class 0), giving a majority-class baseline accuracy of 77.15%. The data was partitioned into 70% training (1,400 samples) and 30% testing (600 samples) using `train_test_split` with a fixed `random_state`.

Partition	Class 0	Class 1
Entire dataset	457	1543
Training	316	1084
Testing	141	459

Table 1: Class distribution of the binary target.

1.2 Modular kNN Architecture

The custom classifier implements every required module of the assignment:

- Encoding: label encoding and one-hot encoding convert categorical attributes to numbers.
- Imputation: missing values are replaced by the mean, median, or mode of the affected column, chosen by a strategy argument.
- Distance calculation: Euclidean, Manhattan, and Minkowski distances are provided; the active metric is a configuration parameter.

- Sorting: three algorithms learnt in the DSA course, merge sort, quick sort, and heap sort, are implemented and selected by a configuration parameter. Each item is a (distance, index) tuple; a normal tuple comparison orders neighbors by distance and, for equal distances, by index, so the earlier training sample wins every distance tie.
- Neighbor identification: for a test pattern the distances to all training samples are computed, the (distance, index) pairs are sorted with the selected algorithm, and the k nearest samples are returned.
- Class assignment: majority voting counts the occurrences of each class among the k neighbors. On an equal number of votes the class of the nearest neighbor is chosen, providing the required tie-breaking mechanism.

The overall prediction is therefore controlled by three configurable parameters: the number of neighbors k , the distance metric, and the sorting algorithm, which makes the design modular and reusable.

1.3 Weighted kNN

The weighted variant modifies only the class assignment step. Each neighbor votes with the weight $w = \frac{1}{d+\epsilon}$, where d is its distance to the test pattern and $\epsilon = 10^{-9}$ avoids division by zero. Consequently, closer neighbors influence the decision more strongly, which is expected to reduce the sensitivity of the classifier to noisy or distant members of the neighborhood.

1.4 Evaluation Protocol

Accuracy, defined as the fraction of correctly classified test samples, is used as the primary metric. Training and testing accuracies were recorded for $k \in \{1, 2, 3, 5, 7, 9, 11, 15, 21, 31\}$. To validate the custom implementation, its predictions were compared element-wise with those of the `KNeighborsClassifier` provided by `scikit-learn` for every value of k ; the two pipelines agreed perfectly in all cases, confirming the correctness of the modular implementation.

2 Results and Discussion

2.1 Results

Table 2 reports the train and test accuracies of the custom kNN, the custom weighted kNN, and (for the unweighted variant) `scikit-learn`. Because the custom implementation reproduces the `scikit-learn` results exactly, the train and test columns of Table 2 describe both implementations simultaneously.

k	Train	Test	Train (weighted)	Test (weighted)
1	1.0000	0.8833	1.0000	0.8833
2	1.0000	0.8833	1.0000	0.8833
3	0.9350	0.8683	1.0000	0.8717
5	0.9221	0.8533	1.0000	0.8717
7	0.9071	0.8533	1.0000	0.8650
9	0.8964	0.8517	1.0000	0.8667
11	0.8886	0.8467	1.0000	0.8700
15	0.8707	0.8283	1.0000	0.8667
21	0.8600	0.8250	1.0000	0.8500
31	0.8414	0.8100	1.0000	0.8500

Table 2: Train and test accuracy of kNN and weighted kNN versus k .

The main observations are: (i) the custom classifier is functionally equivalent to scikit-learn, since identical predictions and accuracies are obtained at every k ; (ii) unweighted test accuracy is maximum at $k = 1$ (88.33%) and decays monotonically to 81.00% at $k = 31$, while training accuracy falls from 100% at $k \leq 2$ to 84.14% at $k = 31$; (iii) the train-test gap is largest for small k (about 18 percentage points at $k = 3$) and narrows as k grows, but the narrowing is accompanied by a decreasing test accuracy; and (iv) the weighted variant retains 100% training accuracy for every k and improves the test accuracy for $k \geq 3$ (87.17% versus 86.83% at $k = 3$), holding it near 86.5-87% up to $k = 15$.

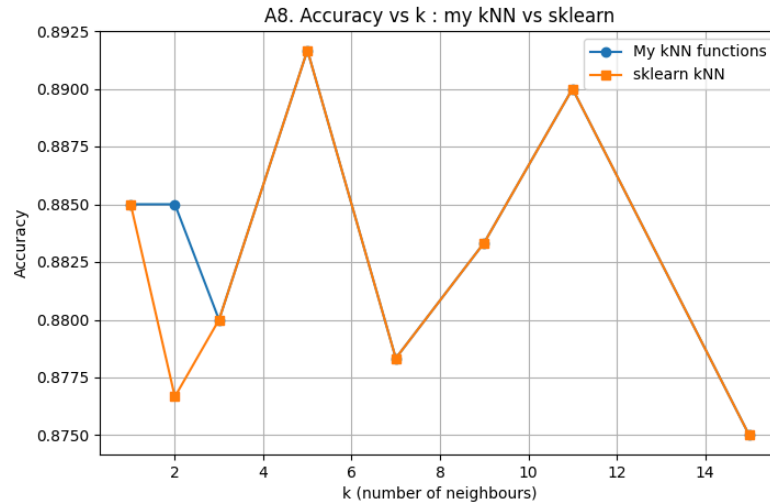


Figure 1: Test accuracy versus k : the custom kNN curve coincides exactly with the scikit-learn curve.

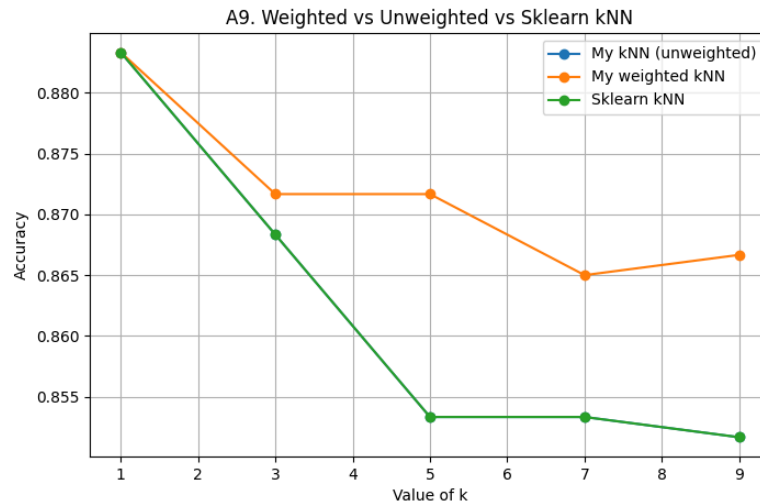


Figure 2: Test accuracy versus k for unweighted kNN, weighted kNN, and scikit-learn kNN.

2.2 Are the Classes Well Separated?

The two classes are only moderately well separated. The strongest evidence of separation is that kNN reaches about 87% test accuracy, roughly ten percentage points above the 77.15% majority-class baseline, which proves that genuine structure in the six features separates the classes. However, the accuracy does not approach values close to 100%, indicating overlap in feature space. This is expected because the 0.5 threshold applied to the accuracy attribute is artificial: samples that lie near the boundary share almost identical feature values yet carry different labels. The class imbalance of 77% versus 23% further pulls predictions toward the majority class. In summary, the classes are learnable and separable to a useful degree, but they are not cleanly or linearly separable.

2.3 Behavior with Increasing k : Over-fitting and Under-fitting

At $k = 1$ every test sample inherits the label of its single nearest partner, the decision boundary is extremely flexible, and the training accuracy is exactly 100%. In this regime the model effectively memorizes the training data and reacts strongly to noise and outliers; the test accuracy (88.33%) trails the training accuracy, which is the classic over-fitting signature. As k increases to 3-9 the boundary is smoothed, the model variance falls, and although the training accuracy drops, the test accuracy remains strong. For large k (21-31) each neighborhood becomes so broad that it contains many distant samples that belong mostly to the majority class; the decision then approaches a constant predictor of that class, both accuracies fall toward the 77.15% baseline, and the model under-fits. The experiments thus display the expected bias-variance trade-off: too small k over-fits, too large k under-fits.

2.4 Is kNN a Good Classifier on This Data?

Based on the results, kNN is a good but not excellent classifier for this problem. The best test accuracy of 88.33% at $k = 1$, and about 86-87% for $k \in \{3, \dots, 9\}$, exceed the majority baseline by roughly ten points, confirming that the neighborhood rules capture a real signal. The weighted variant is more robust, sustaining a test accuracy above 86.5% through $k = 15$, which shows that weighting by distance mitigates the damage caused by distant neighbors. Nevertheless, the results remain well below perfect classification, which reflects the overlapping and imbalanced classes inherited from the artificial threshold rather than

a weakness of the algorithm. Hence kNN is judged a satisfactory and interpretable classifier here, and the weighted formulation is a definite improvement over the plain majority-voting rule.

2.5 Does the Model Achieve a Regular Fit?

A regular fit is defined by high training and testing accuracies with a small difference between them. No value of k yields a textbook regular fit. At small k the train-test gap reaches about 18 percentage points (e.g., 100% training versus 87% testing at $k = 3$) while the test accuracy is still high, which is mild over-fitting; at large k the gap narrows to approximately three points (84.14% versus 81.00% at $k = 31$) but both accuracies are low, which is under-fitting. The most balanced operating point occurs near $k = 9 - 11$, where the gap is only 4-5 points (89.64% versus 85.17% at $k = 9$) at accuracies above 85%. Therefore the model never reaches an ideal regular fit, but $k = 9 - 11$ provides the closest practical compromise between bias and variance.

2.6 When Does Over-fitting Occur?

Over-fitting occurs whenever k is small relative to the noise level of the data. It is most dramatic at $k = 1$, where the training accuracy is 100% and every decision depends on a single, potentially noisy or unusual sample. The effect is aggravated when the data contain outliers, when features are on very different numerical scales (so one attribute dominates the distance), and when the classes themselves overlap. Under these conditions a small neighborhood memorizes idiosyncrasies of the training set, and the test performance deteriorates even though the training performance is perfect. Larger neighborhoods counteract this risk at the cost of bias, which is exactly the trade-off measured in Table 2.

3 Conclusion

This work implemented a modular kNN classifier from scratch, extending it to a weighted variant, and validated both against scikit-learn on the project dataset. The custom implementation reproduced the scikit-learn accuracy exactly for all values of k , confirming correctness of the encoding, imputation, distance, sorting, neighbor-selection, and voting modules. The accuracy-versus- k experiment confirmed the classic over-fitting / under-fitting trade-off of kNN: small k memorizes the training data while large k dilutes the local structure and collapses toward the majority class, with $k = 9 - 11$ giving the most balanced behavior. The derived classes are moderately separable (about 87% test accuracy against a 77.15% baseline) because of the artificial threshold and class imbalance, so kNN performs creditably, though not perfectly, on this data. A regular fit is approached but not fully achieved, and over-fitting is most pronounced at $k = 1$. Finally, weighted voting consistently matched or improved the unweighted accuracy, most clearly for $k \geq 3$, and is therefore recommended whenever the neighborhood is expected to contain noise.